

Lab Report 8: Squeezed Light Generation, Gates, and Measurement

Bryan Fernandez, Pranav Reddy, Nate Barnaby, Gavin Kesler

Electrical and Computer Engineering Department, University of California, Santa Barbara

1 Introduction

This lab explores the generation and transformation of Gaussian states using variable quantum gates. We will start with a vacuum state and apply squeezing, displacement, and rotation operations. Then, we will visualize how each gate alters the state through Wigner function plots, showing how important these gates are to photonic quantum computing. The operations are implemented through integrated optical components. Through simulation, this experiment will improve our understanding of how quantum information is processed through phase space optics.

2 Procedure and Results

Task 1 - Initializing a Vacuum State

To start, we will create a one-mode quantum program to create a vacuum state. The vacuum state represents the lowest energy Gaussian state and is a quantum ground state for variable systems. We will use the strawberryfields library along with the gaussian backend to create the program. The mode was declared and initialized with the Vac operator. The resulting state information will be used for later visualizations.

```
# Task 1 Vacuum State
prog = sf.Program(1)

with prog.context as q:
    Vac | q[0]

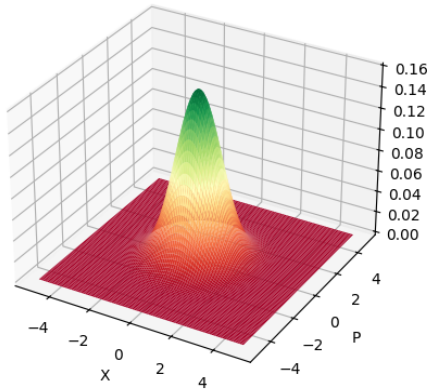
eng = sf.Engine('gaussian')
state = eng.run(prog).state
```

Task 2 - Visualizing Wigner Function

We will plot the Wigner function using matplotlib. The Wigner function is a probability distribution that represents the state in a phase space. It has three axes for: X for position, P for momentum, and Z for probability amplitude.

The Wigner function returns a matrix of values representing the distribution. The plot clearly shows a Gaussian peak centered at the origin. This is typical for a vacuum state. The function smoothly tapers off showing zero mean movement in the phase space. The plot helps us visualize how a quantum state takes space in a phase space.

```
# Task 2
fig = plt.figure()
X = np.linspace(-5, 5, 100)
P = np.linspace(-5, 5, 100)
Z = state.wigner(0, X, P)
X, P = np.meshgrid(X, P)
ax = fig.add_subplot(111,
projection="3d")
ax.plot_surface(X, P, Z, cmap="RdYlGn",
lw=0.5, rstride=1, cstride=1)
fig.set_size_inches(5, 5)
ax.set_axis_on()
ax.set_xlabel('X')
ax.set_ylabel('P')
```



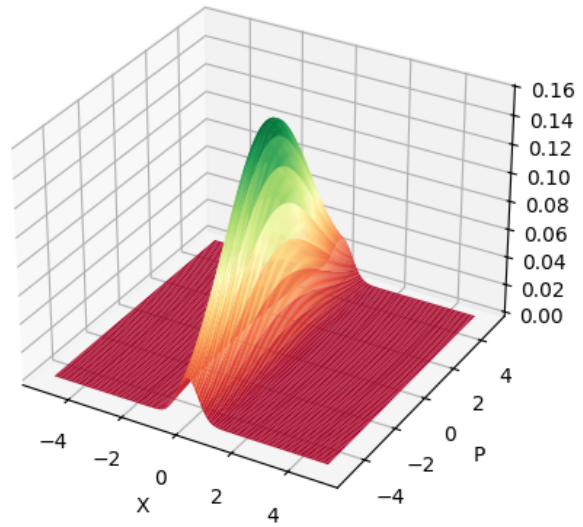
Task 3/4- Squeezing, Rotation, Displacement

To implement the squeezing gate, we create a one mode program initializing the vacuum state. A single mode operation was applied to the mode. The program was executed using the Gaussian backend. The resulting quantum state was stored and used to generate a 3d surface plot of the Wigner function.

```
# Task 3 Squeezing Gate
prog = sf.Program(1)
with prog.context as q:
    S = Sgate(1)
    S | q[0]

eng.reset()
state = eng.run(prog).state
fig = plt.figure()
X = np.linspace(-5, 5, 100)
P = np.linspace(-5, 5, 100)
Z = state.wigner(0, X, P)
X, P = np.meshgrid(X, P)
ax = fig.add_subplot(111, projection="3d")
fig.set_size_inches(5, 5)
ax.plot_surface(X, P, Z, cmap="RdYlGn",
               lw=0.5, rstride=1, cstride=1)
ax.set_axis_on()
ax.set_xlabel('X')
ax.set_ylabel('P')
```

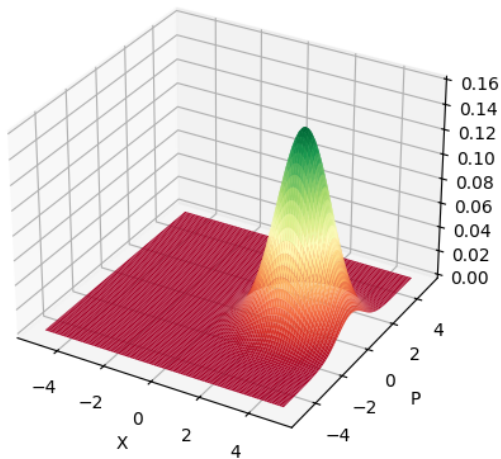
The code successfully produces a Wigner plot that demonstrates the impact of the squeezing operation.



To see the effect of the displacement gate, we once again created a single mode program in the vacuum state. We used the displacement operation to displace the state in the phase space. The simulation was run using the Gaussian backend and visualized. The Wigner function was calculated and plotted in the third dimension.

```
# Displacement Gate
prog = sf.Program(1)
with prog.context as q:
    Dgate(1.5) | q[0]

eng.reset()
state = eng.run(prog).state
fig = plt.figure()
X = np.linspace(-5, 5, 100)
P = np.linspace(-5, 5, 100)
Z = state.wigner(0, X, P)
X, P = np.meshgrid(X, P)
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, P, Z, cmap="RdYlGn",
               lw=0.5, rstride=1, cstride=1)
fig.set_size_inches(5, 5)
ax.set_axis_on()
ax.set_xlabel('X')
ax.set_ylabel('P')
```



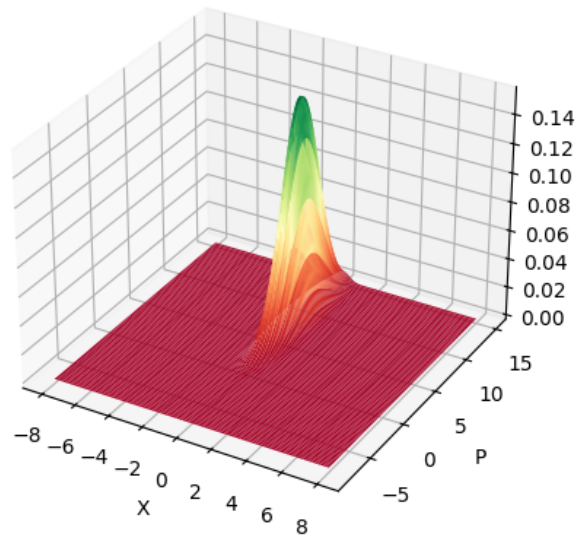
We will once again construct a one mode quantum program with a vacuum state. We will apply three sequential gates. First a displacement gate, followed by a rotation gate with an angle of 90, and finally a squeezing gate. This allows us to observe the transformation of the quantum state in the phase space.

This simulation was executed using the Gaussian backend. The corresponding Wigner function was calculated and visualized in the third dimension.

```
# Rotation Gate
prog = sf.Program(1)
with prog.context as q:
    Dgate(1.5) | q[0]
    Rgate(np.pi/2) | q[0]
    Sgate(1) | q[0]

eng.reset()
state = eng.run(prog).state
fig = plt.figure()
X = np.linspace(-8, 8, 100)
P = np.linspace(-8, 15, 100)
Z = state.wigner(0, X, P)
X, P = np.meshgrid(X, P)
ax = fig.add_subplot(111, projection="3d")
ax.plot_surface(X, P, Z, cmap="RdYlGn",
               lw=0.5, rstride=1, cstride=1)
fig.set_size_inches(5, 5)
ax.set_axis_on()
ax.set_xlabel('X')
```

```
ax.set_ylabel('P')
```



Task 5- 3 combination gates

In the following task, we investigate how ordering the three fundamental single mode operations influences the quantum state. While all three gates are Gaussian operations they are not commutative. This means the final state can be influenced a lot by the sequence they are conducted in.

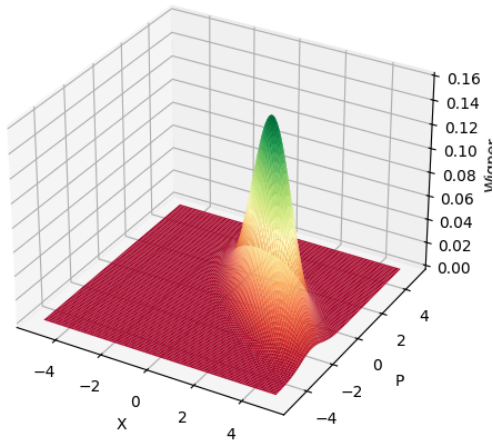
To explore this, we used a code loop that ran the circuit for all six possible orderings of the three gates and visualized the resulting data through Wigner plots as shown previously in the lab report.

These are the following gate sequences were tested:

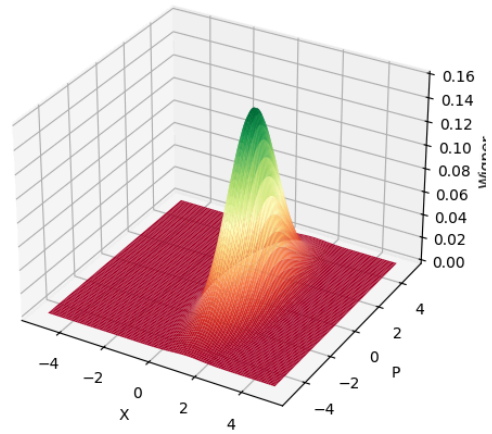
1. S-R-D
2. S-D-R
3. R-S-D
4. R-D-S
5. D-S-R
6. D-R-S
- 7.

The following operations were conducted and the following graphs represent the plots

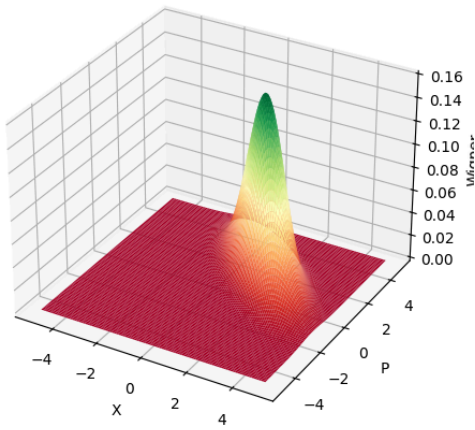
$S \rightarrow R \rightarrow D$



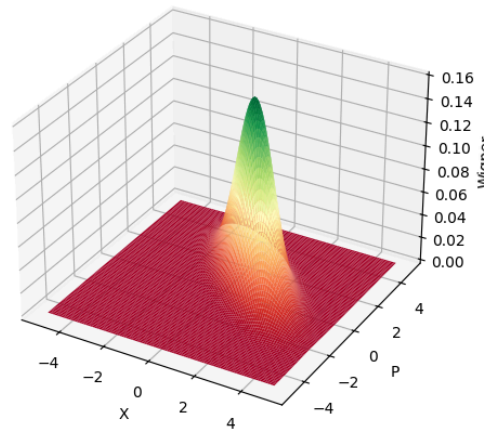
$R \rightarrow D \rightarrow S$



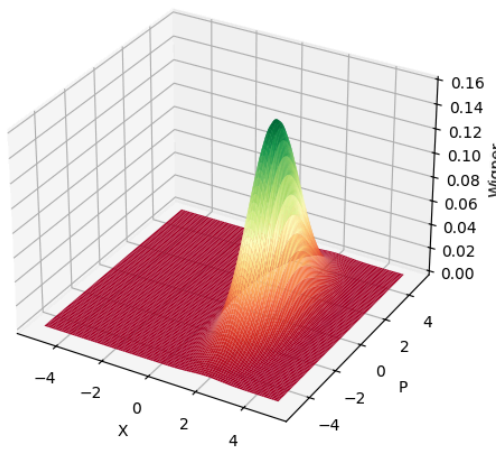
$S \rightarrow D \rightarrow R$



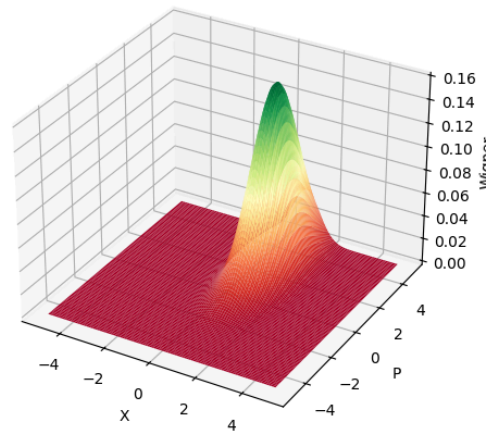
$D \rightarrow S \rightarrow R$



$R \rightarrow S \rightarrow D$



$D \rightarrow R \rightarrow S$



Code for the plots

```
def plot_wigner_for_order(gate_sequence,
title):
    prog = sf.Program(1)
    with prog.context as q:
        for gate in gate_sequence:
            gate | q[0]

    eng = sf.Engine("gaussian")
    result = eng.run(prog)
    state = result.state

    x = np.linspace(-5, 5, 100)
    p = np.linspace(-5, 5, 100)
    X, P = np.meshgrid(x, p)
    Z = state.wigner(0, x, p)

    fig = plt.figure()
    ax = fig.add_subplot(111, projection='3d')
    ax.plot_surface(X, P, Z, cmap='RdYlGn',
linewidth=0.5, rstride=1, cstride=1)
    ax.set_title(title)
    ax.set_xlabel('X')
    ax.set_ylabel('P')
    ax.set_zlabel('Wigner')
    plt.tight_layout()
    plt.show()

squeeze = Sgate(0.5)
displace = Dgate(1.0)
rotate = Rgate(np.pi / 4)

orders = [
    ([squeeze, rotate, displace], "S → R →
D"),
    ([squeeze, displace, rotate], "S → D →
R"),
    ([rotate, squeeze, displace], "R → S →
D"),
    ([rotate, displace, squeeze], "R → D →
S"),
    ([displace, squeeze, rotate], "D → S →
R"),
    ([displace, rotate, squeeze], "D → R → S")
]
```

```
for ops, label in orders:
    plot_wigner_for_order(ops, label)
```

Task 6- Time Domain Ring Circuit

We will explore the time domain multiplexing in the photonic setup. We will implement a TDMProgram in Strawberry Fields. With this we will be able to simulate a looped two mode program that applies a squeezing gate to the mode. Then we will follow it with a beam splitter operation, a rotation gate, and finally a measurement on the looped mode. The parameters used are listed in the code below.

```
# Task 6
# Time Domain Ring Circuit

import strawberryfields as sf
from strawberryfields.ops import Sgate,
BSgate, Rgate, MeasureHomodyne
import numpy as np

def singleloop(r, alpha, phi, theta, shots):
    prog = sf.TDMProgram(N=2)
    with prog.context(alpha, phi, theta) as
(p, q):
        # 1) squeeze the "fresh" temporal mode
q[1]
        Sgate(r, 0) | q[1]
        # 2) interfere the looped mode q[0]
with q[1]
        BSgate(p[0]) | (q[0], q[1])
        # 3) rotate the new mode by φ
Rgate(p[1]) | q[1]
        # 4) homodyne-measure the looped mode
q[0] at angle θ
        MeasureHomodyne(p[2]) | q[0]

    eng = sf.Engine("gaussian")
    result = eng.run(prog, shots=shots)
    # return the homodyne samples for mode 0
    return result.samples_dict[0]
```

Task 7- EPR State Sampling and Variance

The samples collected in Task 6 are analyzed through Task 7 to find the correlation between the homodyne measurements x_0 and x_1 . The data was isolated to measure the results for each mode and then plotted.

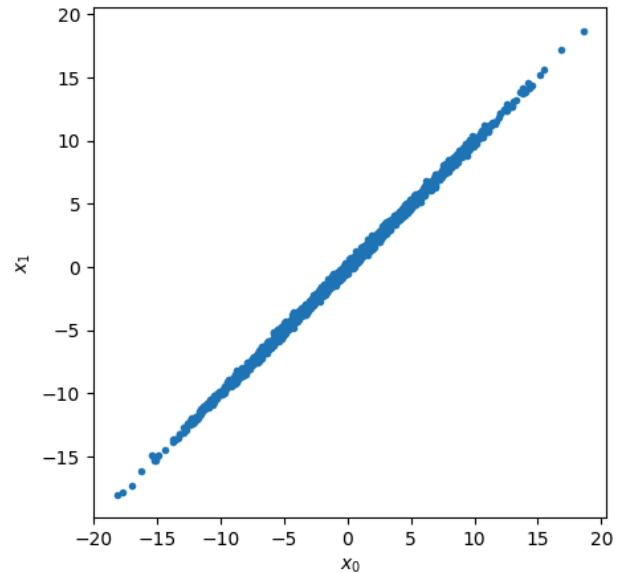
```
# Task 7
r = 2.0
alpha = [np.pi / 4, 0]
phi = [0, np.pi / 2]
theta = [0, 0] # We will measure the
positions by setting theta = [0,0]
shots = 2500

samplesEPRxx = singleloop(r, alpha, phi,
theta, shots)

samplesxx = samplesEPRxx.T
x0 = samplesxx[0]
x1 = samplesxx[1]

# We now want to look at the samples
import matplotlib.pyplot as plt

plt.figure(figsize=(5, 5))
plt.plot(x0, x1, ".")
plt.xlabel(r"$x_0$")
plt.ylabel(r"$x_1$")
plt.show()
```



Sample and Expected Variances:

```
sample_var_xx = np.var(x0 - x1) # from the Monte Carlo samples
sample_var_xx

... np.float64(0.03815936124005136)

expected_var_xx = 2 * np.exp(-2 * r) # theory
expected_var_xx

... np.float64(0.0366312777746836)
```

3 Discussion and Conclusion

This lab through simulations provides us insights to how quantum gate operations affect photonic states. We see this specifically through space visualizations using Wigner functions. Each task builds our knowledge on Gaussian states and transformations by squeezing, displacement, and rotation gates.

In Task 1 and 2, we generated and visualized the vacuum state. The Wigner function appeared symmetric centered around the origin. This showed us the least uncertainty state of light and showed us the basic state we would work with.

In Task 3 and 4, we analyzed the individual effects of quantum gates:

1. The squeezing gate elongated the distribution along one axis. This showed noise suppression in a quadrature and

- amplification in another, demonstrating the key operation in squeezed states.
- 2. The displacement gate shifted the entire Wigner distribution away from the origin, operating like classical amplitude shifts. The shape was not altered.
- 3. The rotation gate as its name suggests rotated the state in the phase space. This transformation was a little harder to notice but while comparing to previous plots its transformation was noticeable.

- Results from measurements agree with theory showing the point of the experiment

In Task 5, we saw how gate order matters. By applying combinations of squeezing, displacement, and rotation in all 6 possible permutations, we saw the resulting states which varied. For example, if we applied a displacement first which would move the distribution away from the origin and then applied the operations it would be different than first squeezing the distribution. This reinforced the idea that quantum operations are not commutative and how even a simple circuit can have complicated behavior due to quantum operator ordering.

In Task 6 and 7, we saw the effects of time domain multiplexing in a resonator. The measured samples seen in a scatter plot showed a strong linear correlation between the two modes. This shows evidence for entanglement. The variance of x_0 and x_1 were close to theoretical predictions validating the simulator's accuracy.

Key Findings:

- Wigner plots serve as strong visual tools to show quantum states in phase states.
- Squeezing, displacement, and rotation affect different aspects of photon distributions
- Gate order matters even in basic circuits
- Time domain multiplexing allows entanglement